The document gives the **correct philosophy and architecture**, but not yet a complete implementation specification.

Right now it tells CTO:

- what ADD-lite is,
- why it exists,
- where the conceptual edge lies,
- what NOT to do.

But you still need one more layer:

exact runtime insertion architecture.

That is the missing piece.

And this is actually the most important question:

> where in the PolyAgora flow does ADD-lite enter?

My strong view:

NOT too early.

Meaning:
ADD-lite should NOT shape:

- regime construction,
- block formation,
- Fixed Star extraction,
- survival matrix generation.

Those must remain:

endogenous structural geometry.

Otherwise:

- circularity enters,
- external semantic noise contaminates the invariant structure,
- the Fixed Star ceases to be internally emergent.

This is critical.

Correct PolyAgora Layering

I think the correct stack is:

Layer 0 — Market Geometry

Raw:

- prices,
- spreads,
- vol,
- correlations,
- etc.

Layer 1 — Internal Polygon Geometry

Current PolyAgora:

- RP coordinates,
- SM,
- local stars,
- block structure,
- admissibility,
- MRTP steering space.

This remains:

internally generated.

Very important.

Layer 2 — ADD-lite External Deformation Layer

Only AFTER:

- block structure exists,
- admissibility exists,
- recoverability exists.

ADD-lite then acts as:

deformation field over steering confidence.

This is the correct location.

Why This Is So Important

If ADD-lite enters too early:
you destroy the purity of:

- Fixed Star emergence,
- endogenous ordering,
- block structure.

Then:
your system becomes:

“another macro-conditioned regime model.”

That would be a mistake.

Correct Runtime Flow

I would define the runtime flow approximately like this:

POLYAGORA FLOW WITH ADD-LITE

STEP 1 — Build Internal Geometry

Compute:

- RP coordinates,
- regime vectors,
- SM,
- local stars,
- block structure,
- admissibility,
- recoverability,
- MRTP paths.

ALL INTERNAL.

No external polygon influence yet.

STEP 2 — Generate Baseline Steering Proposal

PolyAgora computes:

- manifold weights,
- admissibility,
- steering recommendation.

This is:

pure internal topology.

STEP 3 — Compute ADD-lite

Separately compute:

- corridor compression,
- synchronization,
- recoverability degradation,
- path concentration,
  etc.

ADD-lite exists OUTSIDE the endogenous geometry.

This is essential.

STEP 4 — Deform Admissibility / Recoverability

Now ADD-lite modifies:

- confidence,
- aggressiveness,
- maneuverability tolerance.

NOT direction.

Meaning:
ADD-lite says:

- "be less aggressive,"
- "recoverability shrinking,"
- "future-space narrowing."

NOT:

- "go long,"
- "go short."

This distinction is critical.

This Is The Correct Insertion Point

Meaning:
ADD-lite acts:

after regime geometry,

but:

before final allocation execution.

That is exactly the right place.

Conceptually

Internal PolyAgora says:

> "This is the admissible steering structure."

ADD-lite says:

> "Yes, but the future maneuverability space itself is compressing."

That is elegant.

Mathematically

Internal:
$W_i = f(SM, RP, MRTP, A_i, R_i)$

Then external deformation:
$W_i^* = g(ADD_{lite}) \cdot W_i$

or:
$A_i^* = A_i \cdot (1 - ADD_{lite})$

This preserves:

- internal geometry,
- external caution,
- anti-circularity.

What CTO Still Needs

You probably need ONE additional implementation document:

"ADD-lite Runtime Integration Spec"

Containing:

- runtime insertion point,
- update frequency,
- normalization,
- proxy list,
- admissibility deformation logic,
- veto escalation conditions,
- floor/ceiling behavior,
- shadow-testing protocol,
- guardrails.

That is the next document I would create.

My Strong Recommendation

Do NOT let ADD-lite:

- construct regimes,
- construct stars,
- construct blocks,
- define manifold transitions.

Those must remain:

internally emergent topology.

ADD-lite should only:

deform confidence under suspected ordering collapse.

That preserves the beauty of the system.